

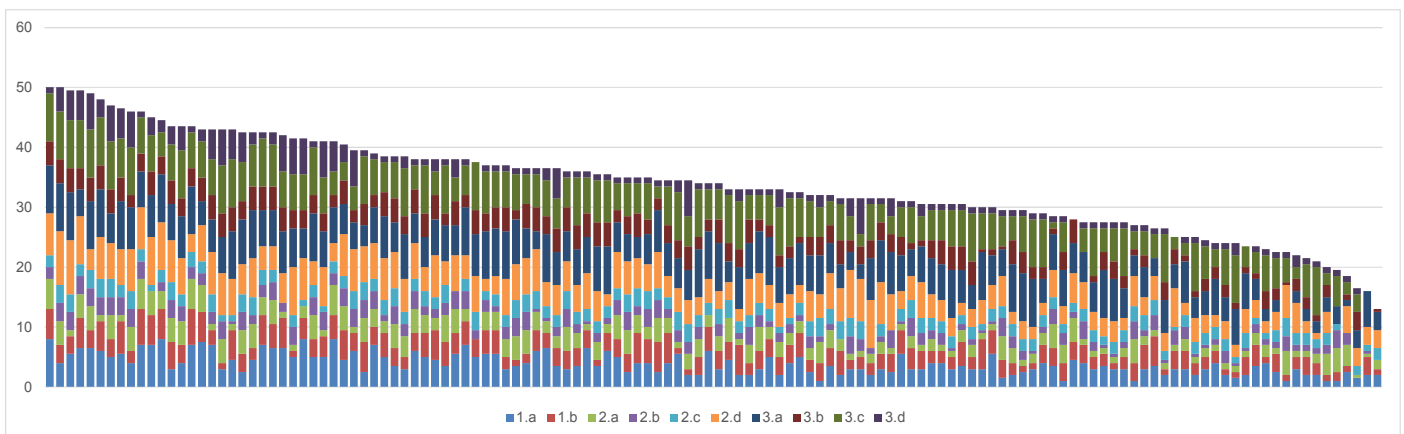
Big Data (H)

Summer 2020 Exam Feedback

The following discusses how well you coped with the exam questions. Key points of the correct answers to the exam questions are also pointed out below; you can also find the exam paper and sample answers on the course's Moodle page. It should be pointed out that, under the no-detriment policy and with the 24-hour release exam mode, it was important that students showcase mastery of the course material in their answers; as such, only the discussion of each answer was marked (i.e., selecting the correct or incorrect option in the MCQ had zero positive or negative weight in the marking scheme) and terse or half-baked copy-paste-like answers did not receive full marks. Markers were also instructed to be quite strict in how they mark, as students had full access to online and offline resources throughout the 24-hour period. These two factors account for most of the marks students lost in this exam, as will be discussed shortly.

An analysis of the results showed that the distribution of marks was almost identical for the undergraduate and PGT cohorts; as such, the figures and statistics below apply to the combined population of both cohorts. Figures/histograms are provided for the sake of clarity and completeness for all subquestions.

The below figure depicts the overall distribution of marks, broken down by question on a per-student basis. Out of all students enrolled in the course, ~18% received a mark in the A's, ~23% received a mark in the B's, ~30% a mark in the C's, ~18% a mark in the D's, and ~10% a mark in the E's or below.

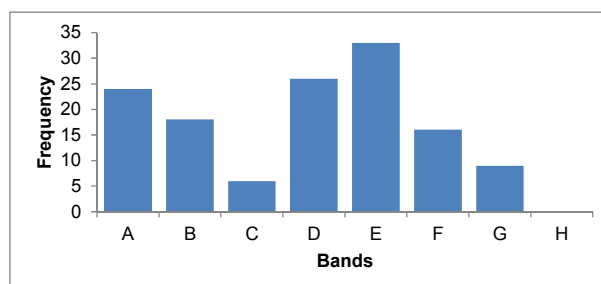


Overall distribution of marks (/60)

Out of the 3 questions, students did on average better in Q3 (max: A1, min: G2, average and median: B3, stddev: ~1.5 bands), less well at Q2 (max: A1, min: F3, average and median: C3, stddev: ~1.5 bands), and even less well in Q1 (max: A1, min: G2, average: D1, median: D2, stddev: ~2 bands).

Question 1:

- A. This was a bookwork/synthesis question, asking students to discuss the reasoning for HDFS's handling of block data and locations, the effects of the design on the performance/scalability of the system, and the requirements it imposes on the namenode (NN)/datanodes (DNs). The question also asked students to explain why incorrect answers were indeed wrong.

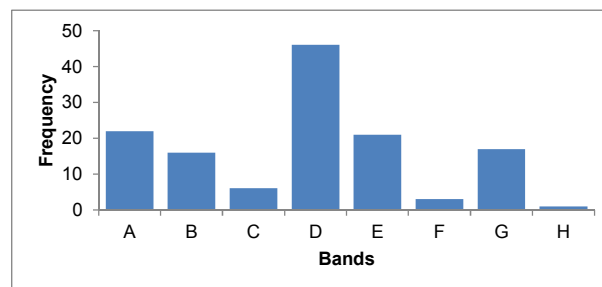


Question 1.a

The correct answer to the MCQ component (not graded) is (b): clients receive the location of blocks from the NN and file blocks from the DNs. There were 2 key points that should have been covered in the answer: (1) by transferring only file locations from the NN (small data transfers), this design takes the NN off the critical path thus allowing the system to scale very well despite the existence of a single mastermind -- the heavy lifting is performed by the large number of DNs that server the actual file data (large data transfers); (2) this design assumes that the NN can serve client requests extremely quickly, requiring that all of the data to be served by the NN fit in memory; this means that either the NN requires an adequately large amount of RAM or there is an upper limit imposed on the size of the filesystem (number of blocks/files). Most students answered this part of the question fairly correctly.

Where several students missed marks was in the explanation of why the other options were incorrect. An answer rehashing what was explained earlier or along the lines of “(c) is incorrect because (b) is correct” did not receive any marks. A correct answer would point out that (a) and (c) move the heavy lifting to the NN, thus rendering it a bottleneck in the system and limiting its scalability. For (d) to be an option it would require that all DNs have a global and consistent view of block status and locations across the whole filesystem; this can be achieved (with broadcast of changes, an epidemic protocol, etc.) -- as a matter of fact, Cassandra does exactly this -- but would trade off the design simplicity of the single mastermind, impose a much higher network overhead and much higher memory requirements on the DNs.

- B. This was another synthesis question, asking students to discuss the disk and memory requirements imposed on the NN and DNs as a result of the system architecture.

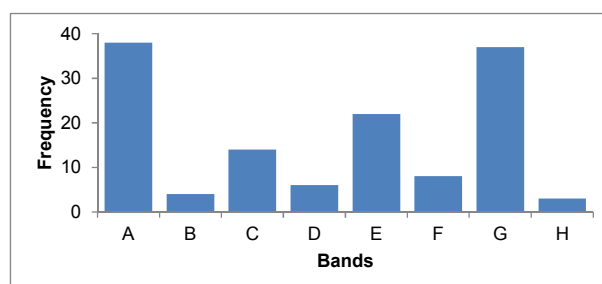


Question 1.b

The correct MCQ choice is (c) in this case. Correct answers would point out that NNs need to store the whole filesystem image in RAM in order to be able to respond to client requests as quickly as possible (in order to keep out of the critical path), hence they require a sufficiently large amount of RAM. On the other hand, NNs only store a journal and snapshot on disk; the journal is periodically purged, the snapshot is a subset of the in-memory file system image (block locations, taking up the majority of space of the in-memory representation, are not stored on disk), and typically hard disks have considerably larger capacities than a machine has RAM. Correct answers would further point out that DNs use RAM mainly for caching filesystem blocks; as such, they would benefit from a larger RAM but that is not critical to their functionality. Sufficiently large disk space, on the other hand, is vital for DNs as they store the bulk of the data handled by the filesystem. Most students answered this question correctly, missing only partial marks for not covering all above key points.

Question 2:

- A. This question was largely a problem solving/synthesis one, where students had to provide the design of a MapReduce job to answer a SELECT DISTINCT query over a key-value dataset.

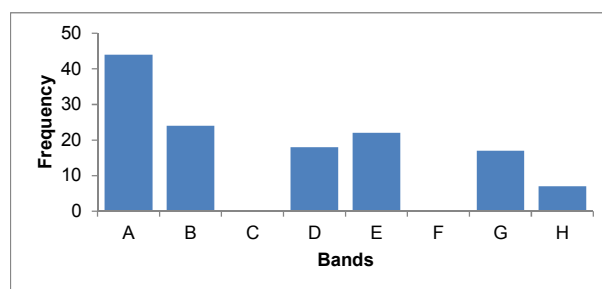


Question 2.a

The correct answer would be based on the fact that MapReduce groups intermediate key-value pairs by key, hence using the combination of data key and data value as the key of the MapReduce key-value pairs would let the MR framework solve the problem for us in a distributed manner. The solution would start by using the TextInputFormat to parse the input file (one line at a time, key: file offset, value: line of text). The mapper's map() function would then (1) tokenise the line to its two numbers, (2) convert them into 32- or 64-bit binary integers and concatenate them into a 64- or 128-bit quantity, and finally (3) emit that as a key with the value set to null. The reducer's reduce() function would simply parse the key into its two components and emit them as an appropriately formatted string, completely ignoring the values.

Most students answered this question correctly, losing only partial marks for missing points (2) and (3); emitting the textual representation and/or a value other than null results in higher memory and bandwidth usage than using the binary representation. Some students missed part of mark for iterating over the values at the reducer side, as that was completely unnecessary (the query was not to count the occurrences of each unique combination). However, there were several students that chose to build a hash table on the mapper or the reducer in order to weed out duplicates; whereas this may be a viable option on the mapper side, it would be an awful idea on the reducer side due to the memory requirements it would impose. Similarly, solutions that emitted the two values of every input line as a key and value (first value as key, second value as value) didn't make use of MapReduce's grouping logic and hence were penalized as well.

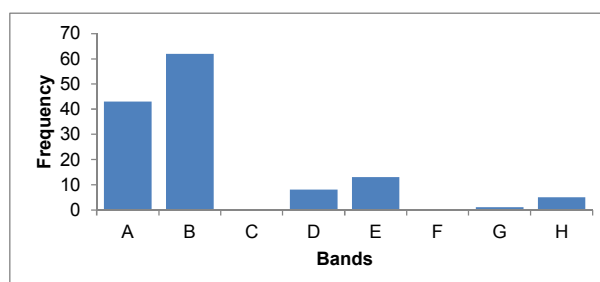
- B. This was also a problem solving question, asking students to revisit their earlier design to use a combiner (to cut down on network transfers and reduce-side processing costs), and to comment on how their earlier solution would have to be amended to accommodate the combiner.



Question 2.b

As the answer to this question was conditional on the design in 2.a, answers were marked on the basis of whether they made sense for the design described by each student in the earlier part. As such, most students received very good marks. Many students missed marks in this question for not answering the final part on whether their design would require changes to be made to either the mapper or reducer. Some few students completely missed the spot and an even smaller number didn't attempt the question at all.

- C. This was a bookwork/synthesis question, asking students to discuss whether reducers can start their execution and/or producing output before all mappers finish, as well as what the repercussions would be on performance.



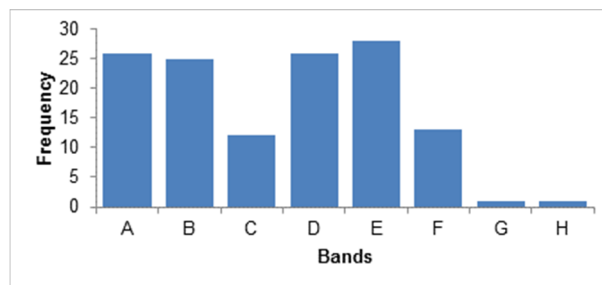
Question 2.c

The correct answer is that reducers can indeed start executing before all mappers finish; this results in amortising the cost of network transfers that are necessary as part of the shuffle phase, with a

trade-off on cluster processing resource being tied up for reducers. Reducers cannot, however, produce output as that would violate the correctness guarantees of the MapReduce pipeline.

Most students did extremely well in this question, as a large part of it is addressed by the course content. Partial marks were lost for not commenting on the performance repercussions, or for obvious misunderstandings on when reducers can actually start transferring data (e.g., a mapper needs to have depleted its input before its output is consolidated and candidate for transfer). A small number of students answered missed points for answering the second part wrong (e.g., by saying that reducers can produce output, but it would then take another reduce round or MapReduce job round to fix any issues, which defeats the whole point of starting the reducers early).

- D. This was a bookwork/synthesis question, asking students to explain how stages are created by Spark's execution engine with regards to wide and narrow RDD partition dependencies, and discuss how and why this design makes sense from a performance/efficiency point of view.



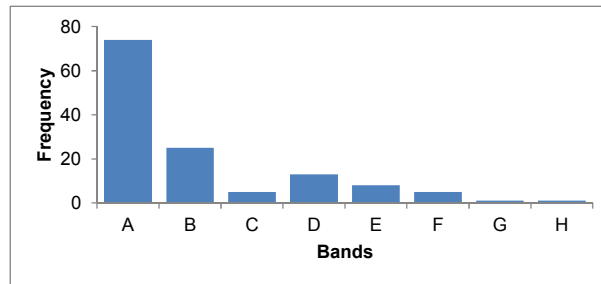
Question 2.d

The correct answer to the first part is that narrow ops are pipelined within a stage, while wide ops signify the boundaries of stages. The correct answer to the second part would then point out that narrow operations are those where a parent RDD partition feeds into a single child RDD partition (or, equivalently, where a child RDD partition is produced based on computation on a single parent RDD partition); this allows these operations to be pipelined -- i.e., executed one after the other -- at the location where the parent RDD partition is located, and as such they incur zero network transfers. In other words, if all it takes to output a key-value in a given partition of RDD Z, is computation on a single key-value from a given partition of RDD Y, which in turn requires computation on a single key-value from a given partition of RDD X, then it makes sense to group said computations so that they are applied in memory one after the other (i.e., grouped and pipelined) to the data of RDD X; as an executor goes through the key-value pairs of a partition of RDD X, it can perform both computations (to produce Y and Z) in one go. This results in major disk/network I/O savings for a negligible memory overhead at the executors (in the general case). On the other hand, wide operations are those where a parent RDD partition feeds into multiple child RDD partitions (and equivalently, each child RDD partition is produced based on computation on multiple parent RDD partitions); this in essence means that wide ops require data to be shuffled around the network, incurring both disk I/O to materialise these partitions and network I/O for the shuffle. As such, narrow ops are pipelined within a stage up to the point where a wide op is found, which then acts as a pipeline breaker and denotes the boundary of that stage.

Most students answered the first part correctly, with some students losing only partial marks for not being clear and/or complete in their discussion. Many students lost marks in the answer to the second part of the question for simply copy-pasting the high-level definition of Spark's stage creation algorithm without getting into the details of how key-values are produced and/or why this separation makes sense from a system/efficiency point of view.

Question 3:

- A. This question was a mix of bookwork and synthesis, asking students to assess the correctness of a number of statements on LSM tree-based data stores and explain their answers.



Question 3.a

Most students did quite well in this question. Breaking it down into individual sub-questions:

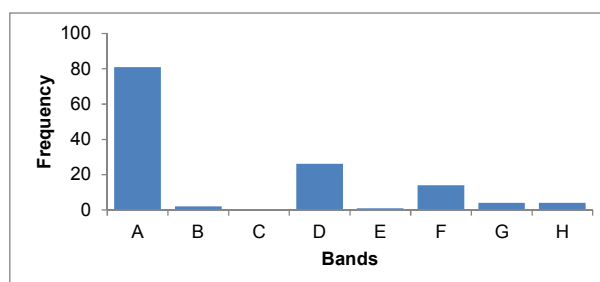
a. With LSM trees read latency goes up as data items are added to the store. The reason is that LSM follows a log structure, always appending new entries to the end of an in-memory buffer, and storing snapshots of said buffer as immutable files on disk. Thus, the more items are added, the more files are created on disk which then need to be scanned to answer an incoming query. A completely correct answer would further point out that compactions, caching and/or other tricks (Bloom filters, per-spill-file indices, etc.) could bring the time down, but in the general case these techniques only partially mitigate the issue; the end result is that read latency is bound to increase. Most students answered this one correctly, losing partial marks for not explaining well enough the reason behind the read latency amplification. Only a few students gave a completely wrong answer.

b. LSM tree-based data stores are indeed expected to exhibit high write throughput. A correct answer would indicate that such data stores first write data in memory (persisting changes in a WaL using group commits to amortise the disk I/O cost), then out to disk to individual immutable files. The combination of in-memory caching and lack of random I/O in both memory and disk, is what gives these systems an edge on write throughput. Although most students picked the correct choice, not many gave a proper explanation of the reason why this is the case (along the above lines).

c. In a non-multi-version LSM-tree-based data store, newer versions of an item hide its old versions (similarly, tombstone records hide previous values of that record), as the system will always return the latest version of any item. Hence, compactions do *not* result in less items being retrievable. With multi-versioning LSM-tree-based data stores, the compaction would not remove versions that are still visible, based on the store's retention policy; as such, again it would not result in retrievable items being deleted. In both cases, at the end of a compaction, there will be less records stored in the compacted files, but the records that were removed would not have been retrievable in the first place. The answers to this subquestion were a mixed bag with just over half students answering the question correctly, while the rest missed the point of older records not being retrievable in the first place.

d. Most students answered this one correctly. The correct answer is that LSM trees are a data structure whose implementation applies on a per-node basis. BigTable/HBase uses LSM trees to manage data in each table partition (tablet/region) on every regionserver; the architecture on top of that (Master, Zookeeper/Chubby, etc.) is only there to facilitate identifying and locating the appropriate tablet/region server. Answers that pointed out that Cassandra follows a peer-to-peer architecture and still its nodes use LSM trees for data management were also taken as correct.

B. This was a synthesis question, asking students to discuss their opinion on whether BigTable/HBase is a rowstore or columnstore.

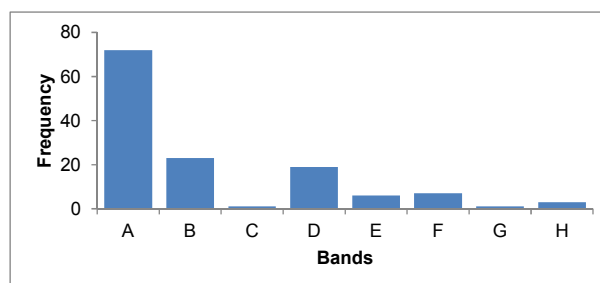


Question 3.b

The correct answer would point out that these systems manage items as key-value pairs where the key is a triplet <rowkey, column identifier, timestamp> and the value is the column value (or equivalently as quadruplets of the form <rowkey, column identifier, timestamp, column value>) and store them sorted by key (i.e., the first three parts of the quadruplet). This is a textbook definition of a rowstore, as all values for a row's columns are stored next to each other. The correct answer would further point out that these systems allow us to group columns into column families, with row data then being split across files in separate directories, one directory per column family. This scheme allows the system to access only data in specific column families without having to touch all of a row's data; in the extreme case where we have a single column per column family, this would be a textbook definition of a column store. As such, these systems are actually a hybrid between the two, offering some of the advantages of both columnstores and rowstores.

Most students did extremely well in this question, as this matter had been discussed multiple times in the lectures. Marks were missed where answers discussed only one of the two aspects. Only very few students gave a completely wrong answer.

- C. This was yet another bookwork/synthesis question, asking students to discuss their views on some properties of the consistent hashing scheme when using an order-preserving hash function. Specifically, breaking it down into sub-questions:



Question 3.c

a. Items would always receive the same identifier under consistent hashing, but they could potentially be assigned to different nodes. The nodes responsible for an item is the node whose identifier exactly succeeds (or precedes, depending on implementation) the item's identifier. Unless an item and a node have the exact same identifiers, it is possible for either new nodes being placed in between an item and an existing node, or for the node currently responsible for an item to go away and hence pass it on to its successor (predecessor). The majority of students answered this question correctly.

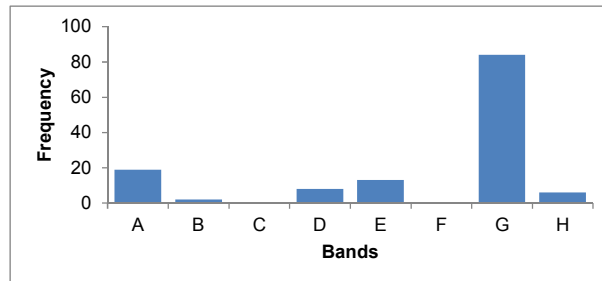
b. The correct answer would point out that, in the general case, the load on nodes caused by an order-preserving hash function on item keys reflects the popularity distribution of values in the key value domain. In other words, ranges of the value domain that are popular will result in the respective node(s) on the data store ring being more loaded than others. Hence, an order preserving hash function would result, in the general case, in an imbalanced load. Again, most students answered this question correctly, but there were several that provided generic answers not taking into account the order-preserving hash function part of the question.

c. The main design requirement behind the use of an order-preserving hash function is to facilitate range queries. The correct answer would point out that, under such a hashing scheme, items with adjacent values are also stored on the same or adjacent nodes. Thus, processing range queries in this setting is a mere matter of identifying the node storing the lower bound of the range, then iteratively hopping from one node to its successor, collecting all relevant data items, until the upper bound of the range is located. The vast majority of students answered this part correctly (including students who gave an incorrect answer to earlier subquestions).

d. Point queries only need to locate a single node; that is, given a key, its hash is computed, then the requested is routed to the single node responsible for that hash (id). The network topology doesn't change depending on the hash function used and as such the choice of hash function has no major effect on the routing/network costs of query processing. If one takes into account the possible load imbalances caused by using an order preserving hash function (as discussed earlier), queries could end up being served by either overloaded or underloaded nodes. In any case, the correct answer

would point out the above and conclude that the choice of hash function will not, in the general case, result in any change in point query performance.

- D. The final question of the exam paper was a synthesis question and a trick one at that. The question asked which combination of read and write quorums can guarantee strong consistency (linearizability) in the face of errors during reads and writes, when using quorum-based replication in a system like Cassandra.



Question 3.d

This was a trick question in that one might think answer (a) is the correct one. However, as discussed in both the lecture and the revision tutorial, this is the case only when one assumes that writes always succeed. The question asks students to consider the situation when there *are* errors during reads and writes, and as such this assumption no longer holds. The correct answer then is that, in the face of errors, one needs to make sure that either all write-quorum replicas are updated, or all are rolled back to their previous state, in an atomic manner in both cases. This requires the use of much stronger/stricter protocols (e.g., Paxos, Raft, etc.). As none of these is used in this setting, the correct answer is then (d). Most students fell in the trap and picked answer (a) as the correct one, then going on to explain how $r+w > N$ plus $w+w > N$ guarantee intersection of the read and write sets, which missed the point of failures breaking this solution. Some students explained that they assumed that (a) is correct when all writes succeed; these answers received partial marks. Few students actually answered the question correctly, pointing out all of the incorrect assumption, the need for a stronger/stricter protocol, and the inability of simple quorum-based replication to achieve strong consistency guarantees.

Last, the Intended Learning Outcomes of the course were assessed by the following assessed exercises and exam questions:

1. *Design, employ/develop and evaluate programs to access big data repositories in a massively parallel manner:* Assessed exercise steps 1 and 2, Assessed in-class quizzes, Exam questions 2A, 2B, 2C and 2D.
2. *Describe and contrast the internals of the design and implementation of current cloud data storage and processing systems:* Assessed exercise steps 1 and 2, Assessed in-class quizzes, All exam questions.
3. *Identify and discuss/analyse issues related to the scalability and efficiency challenges when processing complex queries/algorithms against big data systems, and propose ways of addressing said challenges:* Assessed exercise steps 1 and 2, Assessed in-class quizzes, All exam questions.
4. *Demonstrate that they have mastered the required background knowledge to pursue graduate studies in the fields of cloud systems and big data:* Assessed exercise steps 1 and 2, Assessed in-class quizzes, All exam questions.
5. *(BD (M) only) Explain and appraise state-of-the-art research in the field of big data systems:* Assessed exercise step 2 (benchmarking), Assessed in-class quizzes, All exam questions.